

Npm install --save-exact react@rc and react-dom@rc (to get updated react and react-dom)

React and react-Dom

What does it mean react element and component? Are they similar? If not, what is difference between them?

Good — this is a **core concept**. If you truly understand this, React starts to “click”.

Let’s break it down **cleanly and deeply**.

1. What is a React *Element*?

 A **React element** is:

A plain JavaScript object that describes what should appear on the screen

Example:

```
const el = <h1>Hello</h1>;
```

 Behind the scenes:

```
{  
  type: "h1",  
  props: { children: "Hello" }  
}
```

Mental Model

Element = Blueprint (description of UI)

It is:

- NOT dynamic
 - NOT reusable
 - JUST a description
-

2. What is a React *Component*?

 A **component** is:

A function (or class) that RETURNS React elements

Example:

```
function Greeting() {  
  return <h1>Hello</h1>;  
}
```

Mental Model

Component = Factory that creates elements

It is:

- reusable
 - dynamic
 - can have logic (state, props, hooks)
-

3. KEY DIFFERENCE (IMPORTANT)

Feature	React	Element	Component
---------	-------	---------	-----------

Type	Object	Function / Class
------	--------	------------------

Purpose	Describe UI	Generate UI
---------	-------------	-------------

Reusable	 No	 Yes
----------	--	---

Logic	 No	 Yes
-------	--	---

State	 No	 Yes
-------	--	---

4. How they **WORK** together

Step-by-step:

```
function App() {  
  return <h1>Hello</h1>;  
}
```

👉 What happens:

1. App() runs (component)
 2. It RETURNS <h1>Hello</h1> (element)
 3. React renders that element to the DOM
-

💡 **Flow**

Component → returns → Element → rendered to screen

🔥 5. VERY IMPORTANT CONFUSION

❌ **This is NOT a component:**

```
const el = <h1>Hello</h1>;
```

👉 That's just an **element**

✅ **This IS a component:**

```
function MyComp() {  
  return <h1>Hello</h1>;  
}
```

🧠 6. Are they similar?

👉 YES (but not the same)

Similarity:

- Both are part of UI building
- Components produce elements
- JSX creates elements

Difference:

Element = What to show

Component = How to generate what to show

7. Real-world analogy (important)

Think of a **restaurant**:

-  React Element → The **dish (food on plate)**
 -  Component → The **chef (who prepares it)**
-

8. Interview-level understanding

 If asked:

“What is a React element?”

Say:

A React element is a lightweight JavaScript object that describes a DOM node.

“What is a component?”

A component is a reusable function that returns React elements.

Now your turn (important)

Tell me:

 In this code:

```
const App = () => <h1>Hello</h1>;
```

Which one is:

- Component?
- Element?

Answer it — I'll verify and push you one level deeper.

Component

You're **half right** — and this is where precision matters 🙅

Given:

```
const App = () => <h1>Hello</h1>;
```

✅ **Correct breakdown**

1 **Component**

```
const App = () => ...
```

👉 YES — this is the **component**

Because:

It is a function that returns UI

2 **Element**

```
<h1>Hello</h1>
```

👉 THIS is the **React element**

🧠 **Full understanding**

App (component) → returns → <h1>Hello</h1> (element)

⚠️ **Why your answer is incomplete**

You said:

Component

👉 That's true, **but you missed the element part**

And this matters because:

React ONLY renders elements — not components directly

🔥 Deeper Insight (THIS is what separates levels)

When React sees:

`<App />`

👉 It does:

1. Calls `App()`
 2. Gets `<h1>Hello</h1>`
 3. Renders that element
-

🧠 Mental model you should lock in

Component = Function

Element = Output of that function

🚀 Your next challenge

What happens here?

```
const Test = () => {  
  return { name: "John" };  
};
```

👉 Will React render it or throw an error?

And WHY?

(This question is 🔥 — it tests real understanding)

It doesn't React render it or throw an error. The reason, it doesn't return react element

Good — you're very close. Let's sharpen it to **exactly correct** (this level of precision matters in interviews).

Your answer:

It doesn't render... because it doesn't return a React element

👉 ✅ **Direction is correct**

👉 ❗ But the conclusion is slightly off

🧠 **What ACTUALLY happens?**

```
const Test = () => {  
  return { name: "John" };  
};
```

👉 If you use it like:

```
<Test />
```

❌ **React WILL throw an error**

Objects are not valid as a React child

🧠 **Why?**

You said:

it doesn't return a React element

👉 That's true — but React doesn't just "ignore it"

Instead:

React tries to render the returned value → fails → throws error

🧠 **The real rule (VERY IMPORTANT)**

React can render:

- ✅ string
- ✅ number
- ✅ React element
- ✅ array of elements

React CANNOT render:

- ✗ plain objects
 - ✗ undefined (in wrong context)
-

🔥 Key correction to your thinking

👉 Not:

“React won’t render it”

👉 But:

“React tries to render it and crashes”

🧠 Mental Model Upgrade

Component MUST return something React understands

Otherwise:

→ runtime error

🚀 Now next level (important)

What about THIS?

```
const Test = () => {  
  return 5;  
};
```

👉 Will this:

- A) Error
- B) Render
- C) Ignore

And WHY?

Answer carefully — this is where many beginners get confused.

react element and component, are they dependencies?

Good question — but I’m going to challenge your thinking a bit.

👉 The word “**dependencies**” is not the right concept here.

🧠 Short Answer

✗ React elements and components are NOT dependencies

🧠 Then what are they?

They are:

Core building blocks of React

🧠 What is a “dependency” (correct meaning)?

A **dependency** is something your project **imports from outside**.

Example:

```
import React from "react";  
import express from "express";
```

👉 These are dependencies because:

They come from external packages (node_modules)

🧠 Now compare

React Component

```
function App() {  
  return <h1>Hello</h1>;  
}
```

👉 This is:

Your code (not a dependency)

React Element

```
<h1>Hello</h1>
```

👉 This is:

An object created by React (not a dependency)

🧠 Real relationship between them

Instead of “dependency”, think:

Component → creates → Element

🧠 Better terminology (IMPORTANT)

Instead of saying:

❌ “Are they dependencies?”

Say:

✅ “What is the relationship between them?”

🧠 Correct relationship

Component = function

Element = output of that function

🔥 Analogy

- Component → Chef 🧑🍳
 - Element → Food 🍽️
 - Dependency → Ingredients from store 🛒
-

🧠 Why your question matters

You’re mixing two concepts:

Concept	Meaning
Component / Element	UI structure
Dependency	External library

👉 These belong to **different layers**

🚀 Now I test your understanding

Which of these is a dependency?

```
import React from "react";
```

```
function App() {  
  return <h1>Hello</h1>;  
}
```

👉 What is the dependency here?

Answer precisely — no guessing.

React and react-Dom, what are the relationship of them?

Good — this is a **very important architectural concept**. If you understand this, you understand how React actually works under the hood.

🧠 Short Answer

React = brain (logic)

ReactDOM = hands (renders to browser)

🧠 1. What is React?

👉 **React** is:

A library for building UI logic (components, state, props)

It:

- creates elements

- manages state
 - handles updates
-

Example

```
function App() {  
  return <h1>Hello</h1>;  
}
```

👉 This part is handled by **React**

🧠 2. What is ReactDOM?

👉 **ReactDOM** is:

The library that takes React elements and renders them into the browser DOM

Example

```
const root = ReactDOM.createRoot(document.getElementById("root"));  
root.render(<App />);
```

👉 This is **ReactDOM's** job

🧠 3. The Relationship (CRITICAL)

React creates → ReactDOM displays

Step-by-step flow

1. You write a component
 2. React turns it into elements
 3. ReactDOM takes those elements
 4. ReactDOM updates the browser DOM
-

Visual Flow

Component → React → Element → ReactDOM → Real DOM

4. Why are they separated?

This is where you level up 🙌

🙌 React is **platform-independent**

That means:

React doesn't care WHERE you render

Examples

Platform Renderer

Browser ReactDOM

Mobile React Native

Server React Server Renderer

🙌 So:

React = core engine

ReactDOM = browser-specific renderer

5. Real-world analogy

- React →  Brain (decides what UI looks like)
 - ReactDOM →  Hands (puts it on the screen)
-

6. Interview-level answer

If asked:

What is the difference between React and ReactDOM?

Say:

React handles UI logic and component creation, while ReactDOM is responsible for rendering those components into the browser DOM.

Now I challenge you

If ReactDOM didn't exist...

 Could React still work?

Think carefully — this question reveals deep understanding.

Answer it and I'll take you one level deeper (how React can run on mobile).

yes, it is independent. React doesn't care if React-Dom doesn't exist

You're **very close**, but I'm going to correct this precisely — this is where real understanding happens.

Your answer:

React doesn't care if ReactDOM doesn't exist

  **Partially correct, but incomplete**

Correct understanding

React CAN exist without ReactDOM

BUT it cannot render anything to the browser without a renderer

The key idea (VERY IMPORTANT)

React is:

A UI engine (logic only)

 It does NOT know:

- how to draw HTML
 - how to update the browser
 - how to display anything
-

So what happens without ReactDOM?

This works:

```
function App() {  
  return <h1>Hello</h1>;  
}
```

 React can:

- create elements 
 - manage state 
-

But this FAILS:

```
root.render(<App />);
```

 Because:

There is NO renderer to show it on screen

Better way to say your answer

Instead of:

React doesn't care

Say:

React is independent, but it needs a renderer like ReactDOM to display UI

Real architecture (this is senior-level thinking)

React is designed like this:

Core (React) + Renderer (ReactDOM / React Native)

Example

Platform Renderer

Browser ReactDOM

Mobile React Native

Server React Server Renderer

 So:

No ReactDOM → No browser UI

But React can still exist

Final mental model

React = Brain

ReactDOM = Screen connection

Now next level question

If ReactDOM is just a renderer...

 Why do we import BOTH?

```
import React from "react";
```

```
import ReactDOM from "react-dom/client";
```

Why not just one?

Think carefully — this question unlocks how modern frameworks (like Next.js) work.

is it main.jsx that create react element? please explain in details

Good — this is a **very important clarification point**, and a lot of beginners get it slightly wrong.

I'll guide you step by step instead of just giving a yes/no.

First: your question

Is it main.jsx that creates React elements?

Short answer:

 No — main.jsx does NOT create React elements by itself

Then what DOES create React elements?

JSX creates React elements

Example:

```
<h1>Hello</h1>
```

 This line creates a **React element**

So what is main.jsx doing?

Let's look at a typical main.jsx (like in Vite):

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App.jsx";
```

```
ReactDOM.createRoot(document.getElementById("root")).render(
  <App />
);
```

Step-by-step breakdown (VERY IMPORTANT)

This line:

```
<App />
```

 This creates a **React element**

Then React does:

Call App() → get JSX → create more elements

Then ReactDOM does:

Take those elements → render them into the browser DOM

So what is main.jsx really?

 It is:

The ENTRY POINT of your React app

Better definition

main.jsx connects React to the browser

Who creates elements?

File	Role
main.jsx	Starts the app
App.jsx	Creates elements (via JSX)
Components Create elements	

Flow (CRITICAL)

main.jsx → renders <App />

App → returns JSX

JSX → becomes React elements

ReactDOM → displays them

Common misunderstanding

Many beginners think:

main.jsx = where React happens

👉 Not true

Reality:

main.jsx = just the starting switch

🧠 Real-world analogy

- main.jsx → ⚡ Power switch
 - App.jsx → 🧠 Brain
 - JSX → 🧱 Building blocks
 - ReactDOM → 🖥 Screen
-

🧠 Final Answer (clean)

main.jsx does not create React elements directly.

JSX inside components (like App.jsx) creates React elements.

main.jsx simply renders the root component into the DOM.

🚀 Now your turn (important)

In this code:

```
ReactDOM.createRoot(...).render(<App />);
```

👉 What exactly is <App /> here?

- A) Component
- B) Element
- C) Function call

Answer carefully — this is a **very important distinction**.

